

Where the memory goes

A read of every allocation path in the game: what leaks without bound, what is simply oversized, and what turned out to be fine. Five real leaks — one of them measured at 1.5 GB an hour — one architectural root cause, and about a gigabyte of avoidable steady-state footprint.

repo DmitryAndDemid branch master @ 8c27901 method source read + asset measurement + one field report
 build 0 errors, 97 warnings

1.5 GB/h Managed heap leaked while the attract demo plays — observed, traced to source, and fixed (L1)	~1.2 GB VRAM held at the main menu, before a single stage loads (at 1280×960)	256 MB Allocated by a single dead field with zero references anywhere in the codebase
--	---	---

Part A — Leaks

Things that grow without bound during a normal play session. Ranked by how quickly they accumulate in practice.

L1 CRITICAL FIXED GameBox.cs:1667–1672, 2098 · DEBUG builds

The debug string buffer is never cleared on any backend but Raylib

This is the leak behind the observed 1.5 GB an hour, and it is unrelated to the render-target leaks below: it is plain managed heap, growing every frame the demo draws.

GameBox.RenderBox() appends four interpolated strings per rendered frame:

```

    #if DEBUG
        DebugStrings.Add($"PauseTimestamp: {PauseTimestamp}");
        DebugStrings.Add($"CountTimeFrom: {CountTimeFrom}");
        DebugStrings.Add($"Box Time: {GetTime()}");
        DebugStrings.Add($"Raylib Time: {GetTime()}");
    #endif

```

and GameplayScreenEffect.ApplyShading (GameplayScreenEffect.cs:60–61) adds two more *per active screen effect, per frame*. DebugStrings is a plain List<string> with exactly two clear sites in the tree — GameplayScreen.DrawImGui() (GameplayScreen.cs:568) and PauseMenu.DrawImGui() (PauseMenu.cs:254). Both are DrawImGui overrides, so both die to the same gate — and during a demo **neither one can run**.

Runtime.Render() gates the whole ImGui pass on Engine.Backend.SupportsDebugUi (Runtime.cs:822), and Raylib is the only backend that returns true:

BACKEND	SUPPORTSDEBUGUI	DEBUGSTRINGS CLEARED?
RaylibBackend	true	every frame — no leak
SilkGLBackend	false	never
VulkanBackend	false	never
Metal / SDL / Deko3d	false	never

The shipped `config.json` selects `"Renderer": "vulkan"`, which `Runtime.Start` further redirects to `silk` when an NVIDIA driver file is present — so both routes land on a backend that never clears. The pause menu's clear cannot cover for it either:

`GameplayScreen.TopUpdate` returns early on the `IsDemo` branch before any pause handling, so a demo can never be paused.

Why the rate is so high. The multiplier is grazing. `GameBox.cs:747` spawns a `GrazeScreenEffect` per bullet grazed with a one-second lifetime, so dense patterns keep dozens alive at once, each contributing two strings per frame. At x64 string sizing (22 B + 2 B/char, 8-aligned) plus one 8-byte list slot each:

```
bytes/sec ≈ FPS × (312 + 160 × live_effects)

60 FPS, ~40 live graze effects → 403 KB/s → 1.42 GB/h
144 FPS, ~15 live graze effects → 390 KB/s → 1.37 GB/h
```

It compounds, too: after an hour the list holds roughly 16 million references, so its backing array is a ~128 MB large-object-heap allocation that got there by repeatedly doubling and copying.

CONFIRMING IT

Set `"Renderer": "raylib"` in `config.json` and run the same demo. Raylib is the only backend that calls `DrawImGui()`, so the growth should vanish outright. That separates this from the render-target leaks below, which persist on every renderer.

FIX — APPLIED

`RenderBox()` now clears the buffer at the top of every frame, under `#if DEBUG`, and both reader-side `Clear()` calls are gone. Ownership of the lifetime sits with the producer, so it no longer depends on whether a backend draws an overlay. The four producers route through a new `GameBox.DebugLog` capped at 512 lines, which emits one `... truncated` marker on reaching the cap — so a future change that adds a producer on a path skipping the clear degrades visibly instead of consuming the machine. Both Debug and Release build clean with no new warnings.

TWO NEIGHBOURS WORTH NOTING

`SetTargetFps` is an empty method on both Vulkan (`VulkanBackend.cs:2095`) and Silk (`SilkGLBackend.cs:238`), so `"FrameCap"` does nothing on those backends — frame rate, and therefore this leak's rate, is whatever FIFO vsync gives. And `GameplayScreenEffect` overrides `Equals` without `GetHashCode`, which is among the 97 build warnings.

Removing a screen never frees the GPU resources it owns

`Screen` implements `IDisposable` and defines a virtual `Unload()`; nine screens override it to free render targets. But `RefreshScreens()` — the only place a screen actually leaves

the stack — just drops the reference:

```
Screens.AddRange(QueueToAdd);
QueueToAdd.Clear();
Screens.RemoveAll(x => QueueToRemove.Contains(x)); // no Unload(), no Dispose()
QueueToRemove.Clear();
```

Across roughly 45 `RemoveScreen(...)` call sites, exactly two also call `Unload()` by hand — `PauseMenu.RestartRun` (`PauseMenu.cs:134`) and `CreditsScreen` (`CreditsScreen.cs:270`). Every other teardown path leaks. Because these are GPU framebuffer objects and not managed memory, the GC cannot help: the backend's `Targets / Textures` dictionaries hold the native handles alive for the life of the process.

The leaking screens and what each one abandons, at a 1280×960 window:

SCREEN	RENDER TARGETS ABANDONED	PER TEARDOWN
GameplayScreen + GameBox	4× effects 384×448, Background, Box, UIAboveGameplay, UILeft, score textures	≈ 8.2 MB
TiledLoadingScreen	LoadingBuffer + SwapFrom, both full-window	≈ 9.4 MB
ListSelectScreen	BackgroundCapture, full-window	≈ 4.7 MB
PersonSelectScreen	per-character portrait targets	varies
SpellPracticeDifficultyScreen	4× baked info-text targets	small
IngameSaveReplayScreen	baked text targets	small
GameplayEditorScreen (DEBUG)	3× 8192², plus preview and buffer targets	≈ 805 MB

FIX
Call `screen.Dispose()` (which already routes to `Unload()`) inside `RefreshScreens` as each screen is removed, then delete the two hand-rolled `Unload()` calls so nothing double-frees. Guard against a screen being removed twice in one frame — `PauseMenu` and `IngameSaveReplayScreen` both queue `GameplayScreen` for removal.

L3

CRITICAL Screens/MainScreen.cs:185–213 · consequence of L2

Attract mode leaks a whole GameBox every twenty idle seconds

This is L2's worst instance, and the only one that fires with nobody touching the game. Sit on the title screen and `MainScreen.StartDemo()` builds a full `GameplayScreen` — new `GameBox`, new player, new stage, six render targets — to play a replay as a demo. It ends on any input, on death, or on a clear, via `RemoveScreen(this)` at `GameplayScreen.cs:246`. No `Unload()`. Then the idle timer starts again.

Each cycle strands roughly 8 MB of framebuffers plus the managed `GameBox` graph behind them. Nothing ever reclaims it, so a machine left on the title screen climbs until the driver refuses an allocation.

NOTE
Fixing L2 fixes this one outright. It is worth reproducing first, though — a title screen left running for an hour is the cheapest way to confirm the fix actually took.

L4

CRITICAL Common/MenuScreen.cs:886 · zero callers

Menu item textures are never disposed

Every `MenuItem` bakes its label into its own render texture, and correctly frees the old one when the label changes (MenuScreen.cs:806). It also implements `Dispose()` to free the final one. Nothing calls it — `MenuScreen` holds `List<MenuItem> MenuItems` and defines no `Unload()` at all, so the items are not even reachable from the teardown path that L2 is missing.

The cost per item is modest, but it is paid every single time a menu screen is constructed. Options screens, difficulty select, the music room, the pause menu and the trophy list are all opened and closed repeatedly in one session.

FIX

Give `MenuScreen` an `Unload()` override that disposes every item and clears the list. This needs L2 fixed to have any effect.

L5

MODERATE Screens/ListSelectScreen.cs:66–67

The background capture is reallocated without freeing the old one

The size-change branch drops the previous target on the floor:

```
if (BackgroundCapture == null
    || BackgroundCapture.Value.Texture.Width != Runtime.CurrentRuntime.Width
    || BackgroundCapture.Value.Texture.Height != Runtime.CurrentRuntime.Height)
    BackgroundCapture = LoadRenderTexture(...); // old handle never destroyed
```

Narrow in practice: the internal resolution is fixed for the process lifetime, so the size branch only ever runs on first allocation. It is a trap rather than a live leak — if live resolution switching is ever added, this becomes a 4.7 MB leak per change.

FIX

`UnloadRenderTexture` the old handle before reassigning, exactly as `Helper.EnsureRenderTexture` (Helper.cs:359) already does for the spell-card subtitle targets. That helper is the right pattern; reuse it.

L6

LATENT Utils/Helper.cs:657–672 · dead code

DrawTextAliasedRef leaks its supersampling scratch on every call

It allocates `tmp` at 4× the text size, draws through it, and returns without freeing it. Its sibling `DrawTextAliased` handles the same scratch correctly on both build configurations.

Currently harmless — the only reference is a commented-out line in `DrawTextOutlineRef` (Helper.cs:588), whose entire body is also commented out. Both are dead. But the method is `public` and reads like the natural "no-allocation" variant to reach for, which makes it a live trap for whoever needs one next.

FIX

Delete `DrawTextAliasedRef` and `DrawTextOutlineRef` outright, or add the missing `UnloadRenderTexture(tmp)`. Deleting is better — a partly-commented dead pair is worse than nothing.

Part B — Consumption

Not leaks — this memory is allocated once and held deliberately. It is just far larger than the game needs, and most of it is reachable with small changes.

C1 **888 MB** Runtime.cs:505–541 · LoadTextures()

Every texture in the game is decoded and uploaded at startup

`LoadTextures()` walks `Assets/Textures` and uploads all 127 PNGs unconditionally. On disk they are 63 MB; decoded to RGBA8 in VRAM they are 888 MB, and none is ever unloaded. Measured from the PNG headers:

GROUP	USED BY	VRAM
ending_*	Ending screens — seen once, at the end of a run	304.7 MB
manual-1...9	The in-game manual, 2560×1920 each	170.7 MB
background	gameplay / practice / staff-roll backdrops, 3840×2880	100.2 MB
everything else	Sprites, bullets, UI, <code>stages.png</code> at 21 MB	312.7 MB
Total		888.3 MB

Roughly 475 MB of that — the endings and the manual — is content the player sees once, or never. It is resident from boot to exit on the title screen, during every stage, and on every machine regardless of VRAM.

FIX, IN INCREASING ORDER OF EFFORT

Downscale the oversized source art first — 3840×2880 backdrops render into a 384×448 playfield and a ≤1920px window, so they are carrying 4× more pixels than can ever be sampled. Then make the manual and ending groups load on demand: `Runtime.Textures` is already a dictionary keyed by filename, so a lazy resolver behind it needs no call-site changes. The `TextureManifest` DEBUG consistency check (Runtime.cs:533) will need to learn about deferred keys.

C2 **256 MB** Utils/Helper.cs:655

A dead 8192×8192 scratch target

```
private static TargetHandle AliasTextureTemp = LoadRenderTexture(8192, 8192);
```

A static field initializer, so it allocates the moment anything touches `Helper` — which is during `Load()`, before the menu appears. $8192 \times 8192 \times 4$ bytes = **256 MB** of VRAM.

A full-tree search finds exactly one occurrence of the identifier: the declaration itself. It is written to by nothing, read by nothing, and freed by nothing. It is very likely a leftover from an earlier version of the text-aliasing path that `DrawTextAliased` replaced with per-call scratches.

FIX

Delete the line. Single largest win in the report per character changed. Worth checking `SubtitleBufferTexture` (Helper.cs:343, 381) in the same pass — it is allocated at 8192 px wide and is likewise never read.

Font atlases are stored RGBA8 when three of the four channels are constant

`stb_truetype` rasterises into an 8-bit coverage atlas. The backend then expands it to RGBA, writing 255 into R, G and B for every texel and the real coverage into A, and uploads that:

```
byte[] rgba = new byte[atlasWidth * atlasHeight * 4];
for (int i = 0; i < pixels.Length; i++) {
    rgba[i*4+0] = 255; rgba[i*4+1] = 255; rgba[i*4+2] = 255; // constant
    rgba[i*4+3] = pixels[i];                                // the only real data
}
```

Fonts load at `64 × ScaleF`, which is 128 px at a 1280-wide window, and the atlas sizing loop rounds that up to 2048². That is 16 MB per face; the game ships five, so 80 MB where 20 MB of single-channel data would carry identical information.

FIX

Upload as `R8` and sample `.r` as alpha in the text shader. Roughly 60 MB back across three backends. If that shader change is more disruptive than it is worth, the cheaper half of the win is to stop rounding the atlas up to a power of two — the glyphs occupy about 1300 px of a 2048 px square.

The gameplay editor allocates three 8192² targets on open

`TexturePreview`, `TexturePreview2` and `TextureTestSpellSubtitle` are each 256 MB. Opening the editor asks the driver for 805 MB in one breath, on top of the ~1.2 GB already resident. On a mid-range GPU that is where allocation starts failing, and a failed `CreateTarget` returns a handle the draw path will happily bind.

Under L2 this is also the largest single leak in the codebase: the editor's `Unload()` at line 857 is never reached.

FIX

Size these to what they actually preview. The largest asset in the game is 3840×2880, and the editor draws its previews scaled into a window pane — 2048² is almost certainly generous. This is a `#if DEBUG` path, so it costs shipped players nothing, but it costs whoever is authoring stages a great deal.

Every bullet is a 1.1 KB heap allocation, and there is no pool

```
public int[] Header = new int[128]; // 512 B + header
public float[] FloatingPoints = new float[128]; // 512 B + header
```

Three separate heap objects per bullet, enemy and collectable, all created on `ObjectsAddQueue.Add` and dropped on `BoxObjects.RemoveAll` (`GameBox.cs:451–452`). A dense spell card holding 2,000 objects is ~2.2 MB of live gen-0/gen-1 data being replaced wholesale several times per second.

This is by design — the flat-array model is what the `.sp` schema files describe, and changing it is out of scope for a memory pass. But it is the game's dominant source of GC churn, and it is the number to watch if frame pacing ever gets blamed on collections. A

free-list that recycles dead `RuntimeObject`s (clearing the two arrays rather than reallocating them) would remove nearly all of it without touching the schema or a single hex index.

MEASURE BEFORE ACTING

`dotnet run -- --bench` already prints max / avg / median GC heap for a headless run of the real simulation (Runtime.cs:240). Take that reading first — if the median is comfortable, this is a non-issue and should stay unchanged.

C6

PER DRAW CALL

Rendering/VulkanBackend.cs:1811-1889

The Vulkan descriptor path allocates managed objects per draw

`WriteDescriptors` builds two `List<>`s, calls `Marshal.AllocHGlobal` once per sampler and once per uniform block, then materialises a `WriteDescriptorSet[]` with `ToArray()` — every draw call, at up to `MaxDrawsPerFrame` per frame.

Not a leak: every pointer is freed in the loop at line 1887 and the lists are collectable. But it is the one hot path in the rendering layer that allocates, and it is the busiest code in the frame. The GL backend does the equivalent work with `stackalloc` (SilkGLBackend.cs:914).

FIX

`stackalloc` the two info structs and reuse a preallocated `WriteDescriptorSet[]` field sized to the maximum binding count. Contained change, no API surface affected.

Part C — Checked and clean

Paths that looked like plausible leak sites and are not, recorded so the next audit does not re-walk them.

- ✓ **Backend handle tables.** `DestroyTarget` in the Raylib, Silk-GL, Vulkan and Metal backends all remove the target, its alias texture entry and the colour attachment together. No orphaned dictionary entries.
- ✓ **Gameplay overlays.** `GameplayOverlay.DrawOverlay` unloads on expiry and `GameBox.Dispose` disposes the survivors. Score, timer and signal overlays each free every target they own — including the 2.75 MB `ScoreTexture` a spell-card clear allocates.
- ✓ **Helper scratch targets.** `DrawBossText`, `DrawChapterTitleText`, `DrawTextOutline`, `DrawTextGradient`, `DrawTimerSplash` and `DrawSpellScore` all free their intermediates. The spell-card subtitle explicitly caches its four part-targets rather than churning them per frame.
- ✓ **Chapter and dialog textures.** `RuntimeChapter.Unload` frees the boss and spell title targets, and `GameBox` calls it at all three chapter-transition sites (`GameBox.cs:1333`, `1339`, `1419`).
- ✓ **Unmanaged interop.** Every `AllocHGlobal` in the tree has a matching `FreeHGlobal` — two in the Vulkan descriptor path, one in the Switch pad-state buffer.
- ✓ **File handles.** Every `BitPackage` open is paired with a `Dispose()`, including the exception path in `FileStageInfo.Load (finally, line 78)`.
- ✓ **Simulation hot loop.** `GameBox` has one LINQ expression in the whole file (line 1062, boss lookup) and no per-tick array allocation. The 60 TPS update path itself is allocation-free.
- ✓ **Audio.** Fourteen sounds, 988 KB total, loaded once. No streaming music path exists, so there is nothing to leave open.

One unrelated finding, noted in passing: `Assets/Textures/object.png` is a JPEG (260×280, JFIF) wearing a `.png` extension. It loads because the decoders sniff content rather than trust the name, but it will break any tooling that does not — and it is why the manifest measurement above covers 127 files rather than 128.

Suggested order

Sequenced by result per unit of risk. The first three are hours of work between them and account for most of

the total.

1 Clear DebugStrings in RenderBox — *done*

The `DebugLog`

clear
moved
to
the
point
of
produc
both
reader
side
clears
are
gone,
and
the
produc
now
go
throug
a
capped

. → 1.5 GB/h recovered

See
L1.

2 Delete `Helper.AlliasTextureTemp`

One `SubtitleBufferTexture`

line,
no
behavi
change
nothin
referer
it.
Check

while → 256 MB, zero risk

you
are
in
the
file.

3 Dispose screens in RefreshScreens

Fixes `MenuScreen.Unload`

L2,

L3

and

—

once

exists `GameplayScreen`

—

L3.

Test

the

paths

that

already

unload

by

hand

(pause

menu

restart

credits

for

double

freel,

and

the

paths

where

two

screen

queue

the

same

for → stops all unbounded growth

removal.

4 Downscale the oversized source art

The → 200–300 MB, no code

3840×2880

backdrops

and

the

nine

2560×1920

manual

pages

carry

several

times

more

pixels

than

any

supported

window

can

sample.

Purely

an

asset

change.

5 Load endings and manual pages on demand

Behind `Runtime.Textures`

the

existing

dictionary `TextureManifest`

so

call

sites

are

untouch

Teach

about → ~475 MB off the resident set

deferred

keys

so

the

DEBUG

boot

check

still

passes.

6 Switch font atlases to R8

Three → ~60 MB

backends
and
one
line
in
the
text
shader.

7 Right-size the editor's preview targets

DEBUG → ~600 MB in authoring builds

only,
but
it is
what
makes
the
editor
unopenable
on a
modest
GPU.

8 Delete the dead *Ref text helpers

Housekeeping a trap

Removes
a
leak
that
is
waiting
for
its
first
caller.

How these numbers were reached. One figure is empirical: the 1.5 GB an hour in L1 was observed during demo playback and is reported here as measured, with the source path and the rate formula reconstructed to match it. L1 is the only finding that has been fixed; everything else below it stands as found. Everything else is a static audit — the game was not run. L1 is `#if DEBUG` only; a Release build compiles those `Add` calls away, so it costs shipped players nothing and costs whoever is developing the game everything. VRAM figures are computed as width × height × 4 from PNG headers and from the literal dimensions at each `LoadRenderTexture` call site, assuming RGBA8 and 1 MB = 2²⁰ bytes; window-relative sizes assume 1280×960, where `Runtime.Scale` is 2. Real driver usage will run somewhat higher (alignment, mip and format padding) and the relative ordering is what matters, not the third digit. The tree builds clean at `8c27901`, so every line quoted is live code. For measured rather than computed numbers, `--bench` reports managed heap for the simulation, and the DEBUG F3 overlay already shows working set alongside a texture-VRAM estimate (Runtime.cs:847) — that estimate excludes render targets, which is precisely the memory Part A is about.